

Tutorato: Intel x86-64 e ARM

Esercizio 1

Tradurre la seguente espressione C in Assembly RISC-V, Intel e ARM.

```
int es_1(int b, int c, int d) {  
    return (b + c) - d;  
}
```

RISC-V (gcc -O1):

```
add a0, a0, a1    % a0 = b + c (usando i registri standard degli argomenti)  
sub a0, a0, a2    % a0 = a0 - d  
ret
```

Intel (gcc -O1):

```
leal (%rdi,%rsi), %eax    % Sfrutta Load Effective Address per sommare b e c in %eax  
subl %edx, %eax           % %eax = %eax - d  
ret
```

ARM (gcc -O1):

```
add r0, r0, r1    % r0 = b + c  
subs r0, r0, r2    % r0 = r0 - d (imposta anche i flag)  
bx lr             % Ritorna al chiamante
```

Esercizio 2

Tradurre la seguente istruzione di accesso in memoria ad un array di `int`.

```
void es_2(int* A, int h) {  
    A[4] = h + A[2];  
}
```

Essendo un array di interi a 32 bit (4 byte), l'indice 2 ha offset 8 e l'indice 4 ha offset 16.

RISC-V (gcc -O1):

```
lw a5, 8(a0)      % Carica A[2]  
add a5, a5, a1     % Somma h  
sw a5, 16(a0)     % Salva in A[4]  
ret
```

Intel (gcc -O1):

```
addl 8(%rdi), %esi    % Somma direttamente A[2] (in memoria) a h (%esi)  
movl %esi, 16(%rdi)   % Salva il risultato in A[4]  
ret
```

ARM (gcc -O1):

```
ldr r3, [r0, #8]    % Carica A[2]
add r3, r3, r1      % Somma h
str r3, [r0, #16]   % Salva in A[4]
bx lr
```

Esercizio 3

Tradurre la seguente operazione sfruttando in modo efficiente le operazioni logiche.

```
int es_3(int b) {
    return b * 8;
}
```

RISC-V (gcc -O1):

```
slli a0, a0, 3    % a0 = b « 3
ret
```

Intel (gcc -O1):

```
leal 0(,%rdi,8), %eax    % Calcola base + indice*8. Moltiplica direttamente %rdi per 8
ret
```

ARM (gcc -O1):

```
lsls r0, r0, #3    % Logical Shift Left di 3 posizioni (impostando i flag)
bx lr
```

Esercizio 4

Tradurre il seguente blocco condizionale **if-else**.

```
int es_4(int i, int j, int g, int h) {
    int f;
    if (i < j) f = g + h;
    else f = g - h;
    return f;
}
```

RISC-V (gcc -O1):

```
bge a0, a1, .L2    % Se i >= j salta all'else
add a0, a2, a3     % f = g + h
ret
.L2:
sub a0, a2, a3     % f = g - h
ret
```

Intel (gcc -O1):

```
leal (%rdx,%rcx), %eax    % Calcola provvisoriamente f = g + h in %eax
```

```

subl %ecx, %edx      % Calcola provvisoriamente  $f = g - h$  in %edx
cmpl %esi, %edi      % Confronta i e j
cmovge %edx, %eax    % Se  $i \geq j$ , sovrascrive %eax con %edx ( $g - h$ )
ret

```

ARM (gcc -O1):

ARM utilizza un blocco ITE (If-Then-Else) per eseguire le istruzioni condizionalmente.

```

cmp r0, r1          % Confronta i e j
ite lt             % Dichiaro blocco: Then (Less Than), Else (Opposto)
addlt r0, r2, r3    % Eseguita se  $i < j$  ( $g + h$ )
subge r0, r2, r3    % Eseguita se  $i \geq j$  ( $g - h$ )
bx lr

```

Esercizio 5

Tradurre il seguente semplice costrutto `if` con incremento.

```
int es_5(int i, int j, int f) {
    if (i == j) f = f + 1;
    return f;
}
```

Intel (gcc -O1):

GCC utilizza `sete` per evitare i salti.

```
cmpl %esi, %edi    % Confronta i e j
sete %al           % Imposta il byte %al a 1 se sono uguali, 0 altrimenti
movzbl %al, %eax   % Estende con zeri il byte in un intero a 32 bit
addl %edx, %eax    % Aggiunge 1 o 0 al valore di f
ret
```

ARM (gcc -O1):

```
cmp r0, r1          % Confronta i e j
it eq               % If Then (Equal)
addeq r2, r2, #1    % Incrementa f solo se i == j
mov r0, r2          % Prepara il valore di ritorno
bx lr
```

Esercizio 6

Tradurre il seguente blocco `if-else` di assegnazione.

```
int es_6(int i, int j, int g, int h) {
    int f;
    if (i < j) f = g;
    else f = h;
    return f;
}
```

Intel (gcc -O1):

```
cmpl %esi, %edi    % Confronta i e j
movl %edx, %eax    % %eax = g (caso base)
cmovge %ecx, %eax  % Se i >= j, %eax = h
ret
```

ARM (gcc -O1):

```
cmp r0, r1          % Confronta i e j
ite ge              % If-Then-Else: Greater or Equal
movge r0, r3        % Se i >= j, f = h
movlt r0, r2        % Altrimenti f = g
bx lr
```

Esercizio 7

Tradurre la seguente operazione condizionale (!=).

```
int es_7(int i, int j) {
    if (i != j) i = i + 5;
    return i;
}
```

Intel (gcc -O1):

```
leal 5(%rdi), %eax    % Pre-calcola i + 5 in %eax
cmpl %esi, %edi       % Confronta i e j
cmovne %esi, %eax     % Se sono uguali (condizione falsa), %eax torna al valore originale
ret
```

ARM (gcc -O1):

```
cmp r0, r1            % Confronta i e j
ite ne                % If Not Equal
addne r0, r0, #5      % Se diversi, i = i + 5
moveq r0, r1          % Se uguali, ripristina il valore originale (selettore)
bx lr
```

Esercizio 8

Tradurre il seguente controllo limite (clamp a 0).

```
int es_8(int b, int c) {
    int a = b + c;
    if (a < 0) a = 0;
    return a;
}
```

Intel (gcc -O1):

```
leal (%rdi,%rsi), %eax    % a = b + c
testl %eax, %eax          % Controlla i flag di a
movl $0, %edx             % %edx = 0
cmovs %edx, %eax          % Se Sign Flag è attivo (a < 0), a = 0
ret
```

ARM (gcc -O1):

Sorprendente ottimizzazione bit a bit per evitare salti e selezioni!

```
add r0, r0, r1           % a = b + c
bic r0, r0, r0, asr #31  % Bit Clear. Se a < 0 (segno 1), asr #31 genera 0xFFFFFFFF, azzerando
r0. Se a >= 0, asr #31 genera 0x00000000, lasciando r0 intatto.
bx lr
```

Esercizio 9

Tradurre il seguente ciclo **while** in assembly.

```

int es_9(int* salva, int i, int k) {
    while (salva[i] == k) {
        i += 1;
    }
    return i;
}

```

Intel (gcc -O1):

```

es_9:
movslq %esi, %rax          % Estende l'indice i a 64 bit
leaq 0(,%rax,4), %r8        % Calcola l'offset i * 4 byte
cmpl (%rdi,%rax,4), %edx    % Controlla subito salva[i] == k
jne .L4                    % Se diversi, salta alla fine
addl $1, %esi              % i += 1
movslq %esi, %rcx          % Estende il nuovo i
movq %rcx, %rax
negq %rax                  % Ottimizzazione dei contatori del ciclo
leaq (%r8,%rax,4), %rax
addq %rax, %rdi
.L3:                       % Corpo interno del ciclo
movq %rcx, %rax
addq $1, %rcx              % Incremento i
cmpl %edx, (%rdi,%rcx,4)    % Controllo salva[i] == k
je .L3                     % Se uguali ripete
ret
.L4:
movl %esi, %eax
ret

```

ARM (gcc -O1):

```

es_9:
mov r3, r0                 % R3 = indirizzo base salva
mov r0, r1                 % R0 terrà traccia di i
lsl ip, r1, #2             % IP = offset in byte (i * 4)
ldr r1, [r3, r1, lsl #2]   % Carica salva[i]
cmp r2, r1                 % Confronta con k
bne .L1                   % Se diversi, salta alla fine
add r3, r3, ip             % Sposta la base all'indirizzo corrente
.L3:
adds r0, r0, #1            % Incrementa i
ldr r1, [r3, #4]!         % Carica e aggiorna puntatore post-incremento
cmp r1, r2                 % Confronta con k
beq .L3                   % Se uguali ripete
.L1:
bx lr

```